

生産ライン研究に使う公開データ 原典・取得内容・JSON への変換と限界

Hiroki Funashima

取得・作成日：2026 年 9 月 21 日

1 何を取得し、何に使えるか

今回、**分岐・合流のある標準問題 3 件**と、**小型実験工場**で記録された**生産ログ 1 件**を取得した。これらから、既存の `mock_line_editor.py` で開ける JSON を 5 個作成した。出典だけを紹介するのではなく、原ファイル、変換プログラム、抽出した観測値、検証結果も同梱している。

データ	作業数	分かること	研究での役割
JACKSON	11	作業時間・先行関係	小規模な分岐・合流の確認
MITCHELL	21	作業時間・先行関係	20 工程程度での比較
HESKIA	28	作業時間・先行関係	計画の上限に近い規模の比較
St.Gallen 実験 工場	10	15 生産分の作業経過時間	平均・ばらつきの推定

標準問題は Scholl の公開配布データ [1]、実験工場ログはデータ作成者 Ronny Seiger による Zenodo 公開記録 [2] から取得した。標準問題の「11 作業」は、工場に設備が 11 台あるという意味ではない。作業と設備を区別することが、この資料の重要な点である。

結論を先に

標準問題は、工程のつながりを変えた計算実験に使える。実験工場ログは、実際に記録された時間からモデルの値を作る練習に使える。ただし、今回のデータだけで**実工場のボトルネック検出精度や生産能力を証明することはできない**。

本書では、原データに書かれた**事実**、集計して求めた**統計量**、計算のために追加した**仮定**を区別する。「15 回の平均が 50.332 秒だった」と「今後も対数正規分布に従う」は別の主張である。

2 原典と、実際に取得したファイル

2.1 標準問題：Scholl の配布データ

配布元は、Armin Scholl が提供する Assembly Line Balancing のサイトである [1]。ZIP 内の説明ファイルには、元の資料として次の書誌情報がある。

Armin Scholl (1993), *Data of Assembly Line Balancing Problems*, Schriften zur Quantitativen Betriebswirtschaftslehre 16/93, TH Darmstadt.

配布ページ：<https://assembly-line-balancing.de/salbp/benchmark-data-sets-1993/>

ZIP の precedence graphs/ から、JACKSON.IN2、MITCHELL.IN2、HESKIA.IN2、README.DOC を取り出した。同梱の README.DOC は名前に反して平文であり、古い IN2 形式の読み方を説明している。本書の出典はこの配布データである。各問題名に対応する歴史的な初出論文を独自に再現した、という意味ではない。

2.2 実測記録：St.Gallen 大学の小型実験工場

Ronny Seiger が 2024 年 12 月 13 日に公開したデータセットである [2]。対象は Fischertechnik Industry 9.0V の **物理的な小型実験工場** であり、商用の大規模工場とは規模が異なる。関連論文は、センサーデータから作業を検出する研究 [3] であり、ボトルネックの正解ラベルを提供する研究ではない。

DOI：<https://doi.org/10.5281/zenodo.14441997>

取得ファイル	使用内容
camunda-process.json	生産の開始・終了、工程定義の版、完了状態
camunda-activity.json	各作業の開始・終了、経過時間、作業 ID
production_process.bpmn	生産工程の正式な順序
公開レコードの API メタデータ	DOI、著者、公開日、利用条件、ファイル照合値

BPMN とは、仕事の順序を図と XML で表す形式である。今回は名前や時刻順から工程を推測せず、この工程定義を読んだ。公開レコードの説明中のファイル名と実際のダウンロード名には違いがあるため、上表は**実際の取得名**である。10 Hz のセンサーログ 2 件は、今回の作業時間抽出には不要なので取得していない。

3 標準問題をどう JSON へ変えたか

3.1 原ファイルの読み方

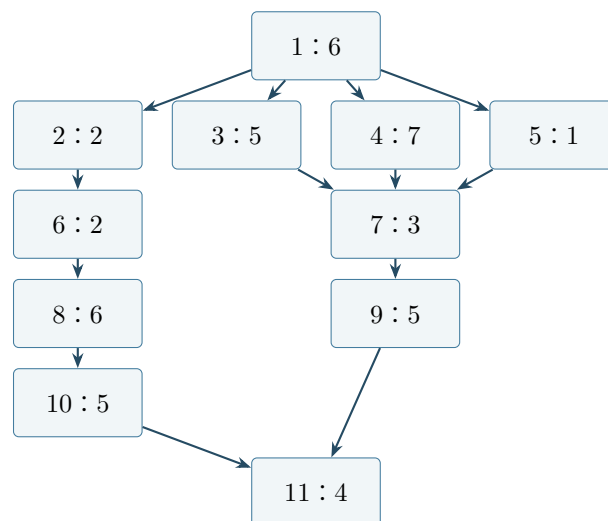
IN2 では、1 行目が作業数 n 、続く n 行が各作業の時間、その後の i, j が「作業 i が終わってから j を始める」という関係である。末尾の $-1, -1$ は終了印であり、工程として取り込まない。作業時間と全接続をそのまま JSON へ転記した。

問題	作業数	接続数	時間の合計	最長作業	1 製品の最短時間 *
JACKSON	11	13	46	7	25
MITCHELL	21	27	105	13	74
HESKIA	28	39	1024	108	467

* 各作業に専用設備を 1 台ずつ与えた今回のモデルでの値。全値は原ファイルからの自作集計。時間単位は IN2 に明記されていないため、秒や分とは決めない。

3.2 JACKSON の接続を実際に見る

次の図は取得した 13 接続をそのまま描いたものである。節点の「番号：時間」は、原ファイルの作業番号と時間を表す。



例えば作業 7 は、作業 $3 \cdot 4 \cdot 5$ の**すべて**が終わるまで開始できない。長い経路は $1 \rightarrow 4 \rightarrow 7 \rightarrow 9 \rightarrow 11$ で、時間は $6 + 7 + 3 + 5 + 4 = 25$ となる。このように最も時間のかかる経路を「クリティカルパス」と呼ぶ。

4 標準問題に追加した仮定

元の SALBP は、複数の作業をいくつかの作業台へ割り当てる問題である [4]。例えば「ねじを締める」と「目視確認」を同じ人が行うこともある。一方、既存プログラムでは、JSON の各工程に独立した設備を与える。この違いを隠すと、元の研究と違う問題を解いていることに気づけなくなる。

項目	原データから採用	今回追加したこと
作業番号・時間	そのまま使用	名前を表示用に付加
先行関係	全接続をそのまま使用	AND 合流として実行
時間のばらつき	反復測定値はない	定数分布、 $p2=0$
設備台数	作業ごとの実設備数は不明	各作業に専用設備 1 台
作業台への割当	元の問題で決める対象	割当の最適化は行わない
投入間隔・生産数	実測値はない	間隔 1、60 製品を既定値にする
画面上の位置	実配置はない	見やすい座標を自動作成
待ち場所・運搬	今回の原ファイルにはない	無限バッファ、運搬時間 0

定数分布とは「毎回同じ時間になる」という設定である。標準偏差を 0 にしたのは、実際の工場にばらつきがないと主張するためではなく、**元データに書かれていないばらつきを作らないため**である。

このデータで言うてよいこと

「公表された作業時間と先行関係を使い、各作業に専用設備を与えた仮想ラインで、待ち時間や改善効果を比較した」と説明できる。「SALBP の最適解を求めた」「実工場の性能を再現した」とは説明できない。

確率的な実験を加える場合には、例えば平均は原値のまま、標準偏差を平均の 15% とする条件を**別ファイル**で作る。この 15% は研究者が設定した条件であり、測定から求めた数値ではない。今回の 3 ファイルには、そのような人工的なばらつきを混ぜていない。

5 実験工場ログから何を抽出したか

5.1 混在した記録を、そのまま平均しない

取得したログには、2022 年から 2023 年にかけての異なる工程版や、請求書処理など別の業務も含まれていた。すべてを一緒に平均すると、「何の作業時間か」が不明になる。そこで、次の条件を**すべて満たす生産**を選んだ。

条件	採用値
工程定義キー	production_process_new3
工程定義の版	2 (定義 ID も完全一致を確認)
開始日	元タイムゾーンで 2023 年 4 月 11 日
状態	COMPLETED、終了時刻あり
作業種別	serviceTask、取消なし、正の経過時間

段階	件数	意味
原ファイル内の生産・業務インスタンス	176	すべての工程定義を含む
対象キーのインスタンス	45	日付や版が異なるものを含む
選択条件に合う完了生産	15	今回の分析対象
その 15 生産に属する全作業・イベント	195	各生産 13 件
自動作業だけを抽出	150	各生産 10 作業

除外した 45 件は、色を入力する人の操作 15 件、開始イベント 15 件、終了イベント 15 件である。原作業ファイル全体は 1727 件だが、その全体を加工時間の標本にはしていない。

5.2 時間と順序の確認

経過時間は `durationInMillis` を 1000 で割り、秒へ直した。150 件すべてについて、終了時刻から開始時刻を引いた値と一致することを確認した。各生産に 10 作業が 1 回ずつあり、BPMN に記された前後関係にも違反がなかった。

同じ名前の「Calibrate VGR」が 2 か所あるため、**作業名ではなく作業 ID** で区別した。小さな値も含め、外れ値の削除や時間の切上げは行っていない。開始・終了時刻、元の記録 ID、JSON 配列内の位置も保存し、後から値を追跡できるようにした。

6 実測から得た 10 工程の値

次の順番は BPMN の自動作業 10 件をたどったものである。今回は一本道であり、分岐・合流はない。日本語名は本資料で付けた表示用の訳で、原名と元の作業 ID は付録・JSON に残している。数値は取得ログからの自作集計であり、論文に掲載された表を転記したものではない。

ID	作業	平均	標準偏差	最小	最大
P01	HBW の原点調整	11.366	3.410	5.953	16.442
P02	VGR の原点調整（前半）	1.012	0.850	0.005	1.764
P03	HBW から取り出す	50.332	6.259	40.508	56.962
P04	炉まで運ぶ	32.464	0.161	32.177	32.907
P05	VGR の原点調整（後半）	5.764	0.024	5.719	5.804
P06	炉で処理する	27.667	0.097	27.538	27.888
P07	WT で運ぶ	32.611	0.998	31.746	34.107
P08	フライス盤を動かす	11.053	0.102	10.942	11.352
P09	仕分ける	28.917	9.401	19.352	47.865
P10	排出口まで運ぶ	36.974	0.763	36.054	38.622

単位は秒。全工程で $n = 15$ 。標準偏差は分母を $n - 1$ とする標本標準偏差である。表示は小数第 3 位までだが、JSON には集計時の精度を保持した。HBW、VGR、WT は原記録の機器略称を残している。

6.1 高校の数学で読める平均とばらつき

15 回の時間を $x_1, \dots, x_{15}$ とすると、平均と標本標準偏差は次のように求める。

$$\bar{x} = \frac{1}{15} \sum_{i=1}^{15} x_i, \quad s = \sqrt{\frac{1}{14} \sum_{i=1}^{15} (x_i - \bar{x})^2}.$$

平均は「全部を同じ時間に均したときの値」で、標準偏差は「値の散らばりの大きさ」を表す。標準偏差は平均の誤差幅そのものではない。

平均が最長なのは P03、秒単位の標準偏差が最大なのは P09 である。しかし、この順位だけで、実工場の生産量を制限するボトルネックを断定することはできない。

短い記録を勝手に直さない

P02 には 0.005–0.011 秒の記録が 6 件、1.643–1.764 秒の記録が 9 件あった。一つの山を持つ滑らかな分布に見えるとは限らない。装置の状態で作業が省略された可能性などを調べる余地があるが、原因はこの集計だけでは分らない。

7 実測由来の JSON を 2 種類にした理由

7.1 まず使う定数版

smart_factory_10_mean.json は、各工程の時間を標本平均で固定する。1 製品を計算すると、10 工程の合計は **238.159 秒**になる。これは 15 生産それぞれの自動作業時間を足し、その合計を平均した値と一致する。

一方、最初の自動作業の開始から最後の自動作業の終了までの実測平均は 238.197 秒である。作業間に 0.020–0.075 秒の小さな隙間があるため、単純な時間の合計とは少し異なる。色を入力する人の作業は、どちらの自動作業時間にも含めていない。

7.2 確率的な計算を試す近似版

smart_factory_10_lognormal.json は、標本平均と標本標準偏差を、対数正規分布の平均・標準偏差として渡す。対数正規分布は、正の時間を出せる分布の一つである。ただし、**この分布が観測値に適しているとは検証したわけではない**。

```
{
  "id": "P03",
  "name": "HBWから取り出す",
  "service": {
    "dist": "lognormal",
    "p1": 50.33233333333333,
    "p2": 6.258984829446233
  },
  "capacity": 1
}
```

配布 JSON の P03 を抜粋。標準偏差も秒単位である。

既存プログラムの p1 と p2 は、**元の秒単位の平均と標準偏差**であり、対数を取った後の値ではない。内部では $m = p1$ 、 $s = p2$ から、

$$\sigma_{\log} = \sqrt{\log\{1 + (s/m)^2\}}, \quad \mu_{\log} = \log m - \frac{\sigma_{\log}^2}{2}$$

へ変換している。

各工程を独立に乱数で作るため、同じ生産内で時間が一緒に増減する関係は再現しない。元の関係を失わないように、smart_factory_observations.json には**生産ごとに 10 作業をまとめた観測値**も保存した。この観測値ファイルは GUI 入力ではなく、将来の分布推定や実測の再解析に使う資料である。

8 既存プログラムで開く方法と編集時の注意

8.1 入力ファイルと既定条件

json/内のファイル	製品数	試行回数	投入間隔
jackson_11.json	60	1	1
mittchell_21.json	60	1	1
heskia_28.json	60	1	1
smart_factory_10_mean.json	1	1	0
smart_factory_10_lognormal.json	1	50	0

全モデルで seed は 42、改善率は 10% である。これらは試行を始めるための**追加設定**で、原典の実験条件を再現したものではない。実測由来の 2 モデルの投入間隔 0 は、1 製品しか投入しない場合の便宜的な値である。標準問題の 1 は原単位、実測由来モデルは秒であり、混同しない。

GUI を起動し、「JSON を開く」から上表のいずれかを選ぶ。工程や接続を編集した後は、新しい名前で保存する。CLI でも同じ JSON を読み込める。

```
python ../mock_line_editor.py run \
  --model json/smart_factory_10_mean.json \
  --out my_results/smart_mean
```

データフォルダから実行し、本体の場所に合わせてパスを変える。出力先は空にする。

8.2 JSON のどの部分を使っているか

キー	内容
meta.version	既存プログラムの形式バージョン 2
nodes	工程 ID、表示名、時間分布、設備台数
edges	前工程 src から後工程 dst への接続
layout	自動作成した画面上の座標
simulation	製品数、試行回数、投入間隔、seed、改善率
meta.sources	本パッケージで追加した出典・仮定

出典を失わずに工程変更を記録する

GUI の再保存では出典情報が残らない。配布 JSON と provenance.json を原本として保持し、変更版は別名で保存する。変更日・変更値・理由も別に記録する。

9 実際に行った検証と、その意味

原ファイルの一致、工程の対応、既存プログラムでの計算を確認した。結果は `validation.json` に保存した。

確認項目	結果
Zenodo ファイルの照合	取得した 3 ファイルの MD5 が公開メタデータと一致
原ファイルの保持	SHA256 を記録。変換時に改変がないか検査
標準問題の転記	3 件、計 60 作業・79 接続を転記
実測の抽出	15 生産・150 作業、各生産で 10 工程が 1 回ずつ
経過時間	150 件で時刻差と一致
工程順序	BPMN との前後関係違反 0 件
モデルの互換性	5 モデルを既存プログラムで読み込み、既定条件で実行
1 製品の計算	標準問題 3 件のクリティカルパスと一致
実測値の直接再生	15 生産すべてで 10 作業の時間合計と一致

9.1 既定条件での計算例

入力	製品数	全完了までの時間	改善効果が最大の作業
JACKSON	60	438	N04
MITCHELL	60	841	N17
HESKIA	60	6839	N13
実測由来・定数	1	238.159 秒	P03
実測由来・対数正規	1	平均 239.190 秒	P03

対数正規版のみ 50 回の平均、ほかは決定的な 1 回の計算である。標準問題 3 件の時間は原単位であり、実測由来の秒とそのまま比較できない。表の順位は**今回の仮想条件下で時間を 10% 短縮したときの順位**である。実工場の正解ラベルではない。

1 製品が一本道を進むモデルでは、どの工程も合計時間に足される。そのため、平均が長い工程を同じ割合で短縮すれば、減る時間も大きくなる。この条件で P03 が選ばれても、「待ち行列のボトルネックを発見できた」という証明にはならない。

検証できた範囲

今回確かめたのは、**データを正しく読めることと、変換・計算の整合性**である。原ファイルの指紋が一致することは、元の計測の正確さや分布の妥当性まで保証するものではない。

10 研究として、何が言えて何が課題か

10.1 今すぐ使える研究の組み立て方

まず JACKSON で分岐・合流の計算を手で追い、MITCHELL と HESKIA へ規模を広げる。次に、投入間隔や、特定工程の時間を変更した別 JSON を作り、「設備待ちが長い工程」と「短縮すると全体が速くなる工程」が一致するかを調べる。原値の JSON は残し、同時に複数の条件を変えすぎないことが比較の助けになる。

実測ログについては、まず 15 生産の時間の範囲や、P02 の二つの群を可視化する。原ログと対応を確認した後で、定数・対数正規・将来追加する経験分布を比較する。経験分布とは、観測した値の中から選び直して時間を作る方法である。既存プログラムは経験分布に未対応なので、今回の JSON を与えるだけではその計算はできない。

10.2 現時点で残る主要な問題

1. **実測対象が小さい。** 小型実験工場の同一日 15 生産であり、商用工場・他製品・別の日へ一般化できる根拠は不足している。
2. **時間の意味が異なる。** ログは制御ソフトから見た作業の経過時間である。指令、通信、装置状態による応答などが含まれ得る。純粋な加工時間、設備待ち、故障時間を分離して測定した値ではない。
3. **設備の共用を表していない。** 同じロボットや装置に関係する複数作業を、既存プログラムでは別の専用設備として扱う。製品を同時に流すと、現実にはできない同時作業を許す可能性がある。
4. **投入と待ちの検証ができない。** 採用した 15 生産同士は時間的に重なっていなかった。ログ全体には貯蔵工程などもあるが、今回のモデルには入れていない。これだけでは、混雑したときの設備待ちや処理能力を検証できない。
5. **確率分布と独立性が未検証である。** P02 のような複数の状態が混ざった値を単一分布へ近似すると、特徴を失う。平均・標準偏差が一致しても、極端に長い時間や工程間の関係が一致するとは限らない。
6. **ボトルネックの正解がない。** どの設備を実際に改善すると生産量が増えたかという介入結果は、今回の入力には含まれない。正解率を評価するには、条件を決めた仮想実験か、追加の実測・改善実験が必要である。

卒業研究では、これらの限界を明示したうえで、「公表構造を使う比較実験」と「実測時間を使うモデル化」の両方を示すと、何を検証したかが明確になる。

11 再現手順、利用条件、参考文献

11.1 元データから作り直す

`convert_sources.py` は Python 標準ライブラリだけで動く。原ファイルの指紋を確認し、同じ抽出・集計から 8 個の JSON を生成する。5 個が GUI 入力、3 個が観測値・出典・一覧の資料である。

```
cd research_data_pack
python convert_sources.py --out rebuilt
python verify_data.py \
  --program ../mock_line_editor.py
lualatex data_sources_guide.tex
lualatex data_sources_guide.tex
```

既存の出力を守るため、変換先は空のフォルダを指定する。検証用プログラムは同梱した Python 本体と、そのコア依存ライブラリを必要とする。LuaLaTeX 原稿には図と表を内蔵し、別の画像や BibTeX ファイルなしで組版できる。同じフォルダの `myfont-setting.sty` が必要で、指定フォントがなければ Latin Modern と原ノ味フォントに切り替える。

11.2 利用条件も出典と一緒に残す

Zenodo の取得メタデータはライセンスを `cc-by-4.0` と示している。原著者・DOI と、抽出・集計・近似を加えたことを記載して使用する。Scholl のサイトは独自の条件を掲示しており、自由な再配布を認める CC BY データとは扱わない [5]。個人の非商用利用や個人へのコピーに関する条件があり、一般公開・再配布の際には掲載条件を確認する。`SOURCE_NOTICES.md` にも取得時点の表示と変更内容を残した。

参考文献

- [1] Armin Scholl. *Data of Assembly Line Balancing Problems*. 1993. Schriften zur Quantitativen Betriebswirtschaftslehre 16/93, TH Darmstadt. 書誌情報は配布 ZIP の README による。配布元：<https://assembly-line-balancing.de/salbp/benchmark-data-sets-1993/> (2026-09-23 取得)。
- [2] Ronny Seiger. *Dataset from a Smart Factory to evaluate a Semi-automated Approach to Detecting Process-Level Activities from Sensor Data*. Zenodo, 2024. <https://doi.org/10.5281/zenodo.14441997> (2026-09-23 取得)。
- [3] Luciano García-Bañuelos, Mauricio Jacobo González González, Ronny Seiger, Marco Franceschetti, Alejandra Guadalupe Silva Trujillo. *A semi-automated approach to detecting process-level activities from sensor data*. *Procedia Computer Science* 257, 856–863, 2025. <https://doi.org/10.1016/j.procs.2025.03.110>.
- [4] Assembly Line Balancing. *SALBP: Problem statement*. <https://assembly-line-balancing.de/salbp/> (2026-09-23 確認)。
- [5] Assembly Line Balancing. *Legal Notice*. <https://assembly-line-balancing.de/home/legal-notice/> (2026-09-23 確認)。

付録 A 実測モデルの ID と原記録の対応

同じ工程を後から追跡できるように、元の作業 ID を保持した。日本語名は表示用であり、作業の識別には ID を使う。

ID	BPMN・ログの作業 ID	原名
P01	Activity_0aproxy	Calibrate HBW
P02	Activity_0cr4src	Calibrate VGR
P03	Activity_1lx6ugd	Unload from HBW
P04	Activity_107pam0	Pickup and transport to Oven
P05	Activity_0jtoxii	Calibrate VGR
P06	Activity_1e6xoye	OV Burn
P07	Activity_075b7q0	WT Transport
P08	Activity_0kh9pn6	Start Milling Machine
P09	Activity_147au71	Sorting Machine sort
P10	Activity_0cwptph	Pickup and transport to sink

A.1 観測値をたどる例

`smart_factory_observations.json` の `cases` には、生産ごとの記録がある。各 `observations` 要素に元の記録 ID、元配列の位置、開始・終了時刻、ミリ秒と秒の値が入る。`source_array_index` は **0 から数える JSON 配列の位置**であり、テキストファイルの行番号ではない。

```
{
  "model_node": "P01",
  "activity_id": "Activity_0aproxy",
  "source_record_id": "Activity_0aproxy:8a11c446-d840-11ed-8f8e-2a1b4c3d6e5f",
  "source_array_index": 105,
  "start_time": "2023-04-11T10:12:02.428+0200",
  "end_time": "2023-04-11T10:12:15.080+0200",
  "duration_millis": 12652,
  "duration_seconds": 12.652
}
```

上は最初の採用生産から取り出した P01 の実測レコードである。完全な生産 ID と、同じ生産の残り 9 作業は配布ファイルを参照する。